# DevLab_MPU6050

1.0.0

Generated by Doxygen 1.17.0

**1 Directory Hierarchy** 1

1.1 Directories . . . 1

**2 Class Index** 3

2.1 Class List . . . 3

**3 File Index** 5

3.1 File List . . . 5

**4 Directory Documentation** 7

4.1 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/basic_↩ Interrupt Directory Reference . . . 7

4.2 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/Basic↩ Read Directory Reference . . . 8

4.3 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples Directory Reference . . . 8

4.4 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C Directory Reference . . . 8

4.5 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/I2↩ CIdentify Directory Reference . . . 9

4.6 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050↩ _BasicRead Directory Reference . . . 9

4.7 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050↩ _ConfigSweep Directory Reference . . . 10

4.8 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050↩ _Interrupt Directory Reference . . . 10

4.9 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src Directory Reference . 11

**5 Class Documentation** 13

5.1 DevLab_MPU6050 Class Reference . . . 13

5.1.1 Detailed Description . . . 14

5.1.2 Member Enumeration Documentation . . . 14

5.1.2.1 InterruptSource . . . 14

5.1.3 Constructor & Destructor Documentation . . . 14

5.1.3.1 DevLab_MPU6050() . . . 14

5.1.4 Member Function Documentation . . . 15

5.1.4.1 begin() . . . 15

5.1.4.2 disableInterrupt() . . . 15

5.1.4.3 enableInterrupt() . . . 16

5.1.4.4 interruptStatus() . . . 16

5.1.4.5 readData() . . . 16

5.1.4.6 sampleRateDiv() . . . 17

5.1.4.7 setAccelScale() . . . 17

5.1.4.8 setClkSource() . . . 17

5.1.4.9 setDLPF() . . . 18

5.1.4.10 setGyroScale() . . . 18

5.1.4.11 setIntPinCfg() [1/2] . . . 18
5.1.4.12 setIntPinCfg() [2/2] . . . 19
5.1.4.13 WhoAmI() . . . 19
5.2 DevLab_MPU6050::interruptPinConfig Struct Reference . . . 19
5.2.1 Detailed Description . . . 20
5.2.2 Member Data Documentation . . . 20
5.2.2.1 activeLow . . . 20
5.2.2.2 clearOnRead . . . 20
5.2.2.3 enableFSyncInterrupt . . . 20
5.2.2.4 enableI2CBypass . . . 20
5.2.2.5 fsyncActiveLow . . . 20
5.2.2.6 latchUntilCleared . . . 21
5.2.2.7 openDrain . . . 21
5.3 DevLab_MPU6050::SensorData Struct Reference . . . 21
5.3.1 Detailed Description . . . 21
5.3.2 Member Data Documentation . . . 21
5.3.2.1 accX . . . 21
5.3.2.2 accY . . . 22
5.3.2.3 accZ . . . 22
5.3.2.4 gyrX . . . 22
5.3.2.5 gyrY . . . 22
5.3.2.6 gyrZ . . . 22
5.3.2.7 temperatureC . . . 22
5.3.2.8 tempRaw . . . 22

**6 File Documentation** **23**

6.1 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/basic_Interrupt/basic_Interrupt.ino File Reference . . . 23
6.2 basic_Interrupt.ino . . . 23
6.3 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/BasicRead/BasicRead.ino File Reference . . . 24
6.4 BasicRead.ino . . . 24
6.5 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/I2CIdentify/I2CIdentify.ino File Reference . . . 25
6.6 I2CIdentify.ino . . . 25
6.7 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_BasicRead/MPU6050_BasicRead.ino File Reference . . . 25
6.8 MPU6050_BasicRead.ino . . . 25
6.9 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_ConfigSweep/MPU6050_ConfigSweep.ino File Reference . . . 26
6.10 MPU6050_ConfigSweep.ino . . . 26
6.11 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_Interrupt/MPU6050_Interrupt.ino File Reference . . . 29
6.12 MPU6050_Interrupt.ino . . . 29

6.13 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.cpp File Reference . . . 30
6.14 DevLab_MPU6050.cpp . . . 30
6.15 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.h File Reference . . . 33
6.15.1 Macro Definition Documentation . . . 34
6.15.1.1 MPU6050_ADDRESS_AD0_HIGH . . . 34
6.15.1.2 MPU6050_ADDRESS_AD0_LOW . . . 34
6.15.1.3 REG_ACCEL_CONFIG . . . 34
6.15.1.4 REG_ACCEL_XOUT_H . . . 35
6.15.1.5 REG_CONFIG . . . 35
6.15.1.6 REG_GYRO_CONFIG . . . 35
6.15.1.7 REG_GYRO_XOUT_H . . . 35
6.15.1.8 REG_INT_ENABLE . . . 35
6.15.1.9 REG_INT_PIN_CFG . . . 35
6.15.1.10 REG_INT_STATUS . . . 35
6.15.1.11 REG_PWR_MGMT_1 . . . 35
6.15.1.12 REG_SMPLRT_DIV . . . 36
6.15.1.13 REG_TEMP_OUT_H . . . 36
6.15.1.14 REG_WHO_AM_I . . . 36
6.15.2 Typedef Documentation . . . 36
6.15.2.1 interruptPinConfig . . . 36
6.15.2.2 SensorData . . . 36
6.15.3 Variable Documentation . . . 36
6.15.3.1 INT_DATA_READY . . . 36
6.15.3.2 INT_FIFO_OVERFLOW . . . 36
6.15.3.3 INT_I2C_MST . . . 37
6.15.3.4 INT_MOTION . . . 37
6.16 DevLab_MPU6050.h . . . 37

**Index** **41**

# Chapter 1

# Directory Hierarchy

## 1.1 Directories


basic_Interrupt . . . 7
basic_Interrupt.ino . . . 23
BasicRead . . . 8
BasicRead.ino . . . 24
examples . . . 8
I2C . . . 8
basic_Interrupt . . . 7
basic_Interrupt.ino . . . 23
BasicRead . . . 8
BasicRead.ino . . . 24
I2CIdentify . . . 9
I2CIdentify.ino . . . 25
MPU6050_BasicRead . . . 9
MPU6050_BasicRead.ino . . . 25
MPU6050_ConfigSweep . . . 10
MPU6050_ConfigSweep.ino . . . 26
MPU6050_Interrupt . . . 10
MPU6050_Interrupt.ino . . . 29
I2C . . . 8
basic_Interrupt . . . 7
basic_Interrupt.ino . . . 23
BasicRead . . . 8
BasicRead.ino . . . 24
I2CIdentify . . . 9
I2CIdentify.ino . . . 25
MPU6050_BasicRead . . . 9
MPU6050_BasicRead.ino . . . 25
MPU6050_ConfigSweep . . . 10
MPU6050_ConfigSweep.ino . . . 26
MPU6050_Interrupt . . . 10
MPU6050_Interrupt.ino . . . 29
I2CIdentify . . . 9
I2CIdentify.ino . . . 25
MPU6050_BasicRead . . . 9
MPU6050_BasicRead.ino . . . 25

MPU6050_ConfigSweep . . . . . 10
MPU6050_ConfigSweep.ino . . . . . 26
MPU6050_Interrupt . . . . . 10
MPU6050_Interrupt.ino . . . . . 29
src . . . . . 11
DevLab_MPU6050.cpp . . . . . 30
DevLab_MPU6050.h . . . . . 33

# Chapter 2

# Class Index

## 2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:


DevLab_MPU6050 . . . . . 13
DevLab_MPU6050::interruptPinConfig . . . . . 19
DevLab_MPU6050::SensorData . . . . . 21

# Chapter 3

# File Index

## 3.1 File List

Here is a list of all files with brief descriptions:

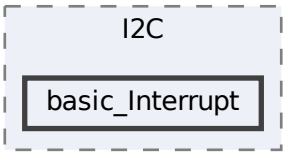

/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/basic_↩
Interrupt/basic_Interrupt.ino . . . . . 23
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/BasicRead/↩
BasicRead.ino . . . . . 24
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/I2CIdentify/↩
I2CIdentify.ino . . . . . 25
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_↩
BasicRead/MPU6050_BasicRead.ino . . . . . 25
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_↩
ConfigSweep/MPU6050_ConfigSweep.ino . . . . . 26
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_↩
Interrupt/MPU6050_Interrupt.ino . . . . . 29
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.cpp 30
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.h . . 33

# Chapter 4

# Directory Documentation

## 4.1 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/basic_Interrupt Directory Reference

Directory dependency graph for basic_Interrupt:

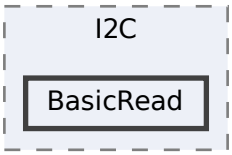



### Files

- file basic_Interrupt.ino

## 4.2 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/BasicRead Directory Reference

Directory dependency graph for BasicRead:




### Files

- file BasicRead.ino

## 4.3 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples Directory Reference

### Directories

- directory I2C

## 4.4 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C Directory Reference

Directory dependency graph for I2C:

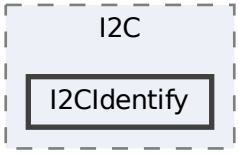



### Directories

- directory basic_Interrupt
- directory BasicRead
- directory I2CIdentify
- directory MPU6050_BasicRead
- directory MPU6050_ConfigSweep
- directory MPU6050_Interrupt

## 4.5 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/I2CIdentify Directory Reference

Directory dependency graph for I2CIdentify:

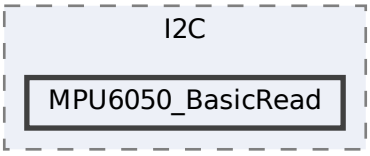



**Files**

- file I2CIdentify.ino

## 4.6 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/MPU6050_BasicRead Directory Reference

Directory dependency graph for MPU6050_BasicRead:

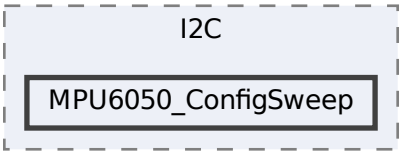



**Files**

- file MPU6050_BasicRead.ino

## 4.7 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/MPU6050_ConfigSweep Directory Reference

Directory dependency graph for MPU6050_ConfigSweep:

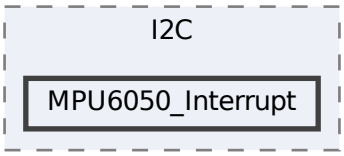



**Files**

- file MPU6050_ConfigSweep.ino

## 4.8 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/MPU6050_Interrupt Directory Reference

Directory dependency graph for MPU6050_Interrupt:




**Files**

- file MPU6050_Interrupt.ino

# 4.9 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/src Directory Reference

### Files

- file DevLab_MPU6050.cpp
- file DevLab_MPU6050.h

# Chapter 5

# Class Documentation

## 5.1 DevLab_MPU6050 Class Reference

`#include <DevLab_MPU6050.h>`

### Classes

- struct SensorData
- struct interruptPinConfig

### Public Types

- enum InterruptSource : uint8_t { INT_DATA_READY = 0x01 , INT_I2C_MST = 0x08 , INT_FIFO_OVERFLOW = 0x10 , INT_MOTION = 0x40 }

### Public Member Functions

- DevLab_MPU6050 (TwoWire &wire=Wire, uint8_t address=0x68)

  *Construct a new DevLab_MPU6050 object.*
- bool begin (uint8_t sdaPin, uint8_t sclPin, uint32_t clock=400000)

  *Initialize the MPU6050.*
- bool readData (SensorData &dataOut)

  *Read the sensor data.*
- void setAccelScale (uint8_t scale)

  *Set the accelerometer scale.*
- void setGyroScale (uint8_t scale)

  *Set the gyroscope scale.*
- uint8_t WhoAmI ()

  *Get the device ID.*
- void setDLPF (uint8_t dlpSetting)

  *Set the Digital Low Pass Filter (DLPF) setting.*
- void sampleRateDiv (uint8_t div)

  *Set the sample rate divider.*
- void setClkSource (uint8_t clkSource)

*Set the clock source.*

- void setIntPinCfg (uint8_t intPinCfg)

  *Set the INT pin / bypass configuration register directly.*

- void setIntPinCfg (const interruptPinConfig &cfg)

  *Set the INT pin / bypass configuration from a struct.*

- void enableInterrupt (uint8_t sources)

  *Enable one or more interrupt sources (OR'd together), leaving others untouched.*

- void disableInterrupt (uint8_t sources)

  *Disable one or more interrupt sources (OR'd together), leaving others untouched.*

- uint8_t interruptStatus ()

  *Read the INT_STATUS register without side effects other than the hardware's own clear-on-read.*

### 5.1.1 Detailed Description

Definition at line 43 of file DevLab_MPU6050.h.

### 5.1.2 Member Enumeration Documentation

#### 5.1.2.1 InterruptSource

```
enum DevLab_MPU6050::InterruptSource : uint8_t
```

**Enumerator**

| | |
|---|---|
| INT_DATA_READY | |
| INT_I2C_MST | |
| INT_FIFO_OVERFLOW | |
| INT_MOTION | |

Definition at line 68 of file DevLab_MPU6050.h.

### 5.1.3 Constructor & Destructor Documentation

#### 5.1.3.1 DevLab_MPU6050()

```
DevLab_MPU6050::DevLab_MPU6050 (
            TwoWire & wire = Wire,
            uint8_t address = 0x68)
```

Construct a new DevLab_MPU6050 object.

**Parameters**

| | |
|---|---|
| *wire* | The I2C bus to use. |
| *address* | The I2C address of the MPU6050. |

Definition at line 14 of file DevLab_MPU6050.cpp.

### 5.1.4 Member Function Documentation

#### 5.1.4.1 begin()

```
bool DevLab_MPU6050::begin (
            uint8_t sdaPin,
            uint8_t sclPin,
            uint32_t clock = 400000)
```

Initialize the MPU6050.

**Parameters**

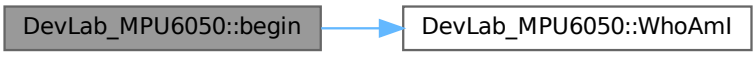
| | |
|---|---|
| *sdaPin* | The SDA pin. |
| *sclPin* | The SCL pin. |
| *clock* | The I2C clock frequency. |

**Returns**

True if the MPU6050 was initialized successfully, false otherwise.

Definition at line 21 of file DevLab_MPU6050.cpp.

Here is the call graph for this function:




#### 5.1.4.2 disableInterrupt()

```
void DevLab_MPU6050::disableInterrupt (
            uint8_t sources)
```

Disable one or more interrupt sources (OR'd together), leaving others untouched.

**Parameters**

| | |
|---|---|
| *sources* | Bitwise-OR of InterruptSource values. |

Definition at line 135 of file DevLab_MPU6050.cpp.

#### 5.1.4.3 enableInterrupt()

```
void DevLab_MPU6050::enableInterrupt (
            uint8_t sources)
```

Enable one or more interrupt sources (OR'd together), leaving others untouched.

**Parameters**

| | |
|---|---|
| *sources* | Bitwise-OR of InterruptSource values. |

Definition at line 130 of file DevLab_MPU6050.cpp.

#### 5.1.4.4 interruptStatus()

```
uint8_t DevLab_MPU6050::interruptStatus ()
```

Read the INT_STATUS register without side effects other than the hardware's own clear-on-read.

**Returns**

The interrupt status byte.

Definition at line 140 of file DevLab_MPU6050.cpp.

#### 5.1.4.5 readData()

```
bool DevLab_MPU6050::readData (
            SensorData & dataOut)
```

Read the sensor data.

**Parameters**

| | |
|---|---|
| *dataOut* | The sensor data to read. |

**Returns**

True if the data was read successfully, false otherwise.

Definition at line 47 of file DevLab_MPU6050.cpp.

#### 5.1.4.6 sampleRateDiv()

```
void DevLab_MPU6050::sampleRateDiv (
            uint8_t div)
```

Set the sample rate divider.

**Parameters**

| | |
|---|---|
| *div* | The sample rate divider to apply. |

Definition at line 103 of file DevLab_MPU6050.cpp.

#### 5.1.4.7 setAccelScale()

```
void DevLab_MPU6050::setAccelScale (
            uint8_t scale)
```

Set the accelerometer scale.

**Parameters**

| | |
|---|---|
| *scale* | The accelerometer scale to apply. |

Definition at line 82 of file DevLab_MPU6050.cpp.

#### 5.1.4.8 setClkSource()

```
void DevLab_MPU6050::setClkSource (
            uint8_t clkSource)
```

Set the clock source.

**Parameters**

| | |
|---|---|
| *clkSource* | The clock source to apply. |

Definition at line 107 of file DevLab_MPU6050.cpp.

#### 5.1.4.9 setDLPF()

```
void DevLab_MPU6050::setDLPF (
            uint8_t dlpSetting)
```

Set the Digital Low Pass Filter (DLPF) setting.

**Parameters**

| | |
|---|---|
| *dlpSetting* | The DLPF setting to apply. |

Definition at line 74 of file DevLab_MPU6050.cpp.

#### 5.1.4.10 setGyroScale()

```
void DevLab_MPU6050::setGyroScale (
            uint8_t scale)
```

Set the gyroscope scale.

**Parameters**

| | |
|---|---|
| *scale* | The gyroscope scale to apply. |

Definition at line 93 of file DevLab_MPU6050.cpp.

#### 5.1.4.11 setIntPinCfg() [1/2]

```
void DevLab_MPU6050::setIntPinCfg (
            const interruptPinConfig & cfg)
```

Set the INT pin / bypass configuration from a struct.

**Parameters**

| | |
|---|---|
| *cfg* | The interrupt pin configuration to apply. |

Definition at line 119 of file DevLab_MPU6050.cpp.

#### 5.1.4.12 setIntPinCfg() [2/2]

```
void DevLab_MPU6050::setIntPinCfg (
            uint8_t intPinCfg)
```

Set the INT pin / bypass configuration register directly.

**Parameters**

| | |
|---|---|
| *intPinCfg* | Raw byte to write to REG_INT_PIN_CFG. |

Definition at line 115 of file DevLab_MPU6050.cpp.

#### 5.1.4.13 WhoAmI()

```
uint8_t DevLab_MPU6050::WhoAmI ()
```

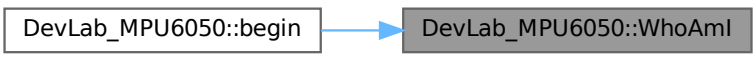
Get the device ID.

**Returns**

The device ID.

Definition at line 43 of file DevLab_MPU6050.cpp.

Here is the caller graph for this function:




The documentation for this class was generated from the following files:

- /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.h
- /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.cpp

## 5.2 DevLab_MPU6050::interruptPinConfig Struct Reference

```
#include <DevLab_MPU6050.h>
```

**Public Attributes**

- bool activeLow = false
- bool openDrain = false
- bool latchUntilCleared = false
- bool clearOnRead = false
- bool fsyncActiveLow = false
- bool enableFSyncInterrupt = false
- bool enableI2CBypass = false

### 5.2.1 Detailed Description

Definition at line 58 of file DevLab_MPU6050.h.

### 5.2.2 Member Data Documentation

#### 5.2.2.1 activeLow

```
bool DevLab_MPU6050::interruptPinConfig::activeLow = false
```

Definition at line 59 of file DevLab_MPU6050.h.

#### 5.2.2.2 clearOnRead

```
bool DevLab_MPU6050::interruptPinConfig::clearOnRead = false
```

Definition at line 62 of file DevLab_MPU6050.h.

#### 5.2.2.3 enableFSyncInterrupt

```
bool DevLab_MPU6050::interruptPinConfig::enableFSyncInterrupt = false
```

Definition at line 64 of file DevLab_MPU6050.h.

#### 5.2.2.4 enableI2CBypass

```
bool DevLab_MPU6050::interruptPinConfig::enableI2CBypass = false
```

Definition at line 65 of file DevLab_MPU6050.h.

#### 5.2.2.5 fsyncActiveLow

```
bool DevLab_MPU6050::interruptPinConfig::fsyncActiveLow = false
```

Definition at line 63 of file DevLab_MPU6050.h.

#### 5.2.2.6 latchUntilCleared

```
bool DevLab_MPU6050::interruptPinConfig::latchUntilCleared = false
```

Definition at line 61 of file DevLab_MPU6050.h.

#### 5.2.2.7 openDrain

```
bool DevLab_MPU6050::interruptPinConfig::openDrain = false
```

Definition at line 60 of file DevLab_MPU6050.h.

The documentation for this struct was generated from the following file:

- /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.h

## 5.3 DevLab_MPU6050::SensorData Struct Reference

```
#include <DevLab_MPU6050.h>
```

### Public Attributes

- int16_t accX
- int16_t accY
- int16_t accZ
- int16_t gyrX
- int16_t gyrY
- int16_t gyrZ
- int16_t tempRaw
- float temperatureC

### 5.3.1 Detailed Description

Definition at line 45 of file DevLab_MPU6050.h.

### 5.3.2 Member Data Documentation

#### 5.3.2.1 accX

```
int16_t DevLab_MPU6050::SensorData::accX
```

Definition at line 46 of file DevLab_MPU6050.h.

#### 5.3.2.2 accY

```
int16_t DevLab_MPU6050::SensorData::accY
```

Definition at line 47 of file DevLab_MPU6050.h.

#### 5.3.2.3 accZ

```
int16_t DevLab_MPU6050::SensorData::accZ
```

Definition at line 48 of file DevLab_MPU6050.h.

#### 5.3.2.4 gyrX

```
int16_t DevLab_MPU6050::SensorData::gyrX
```

Definition at line 50 of file DevLab_MPU6050.h.

#### 5.3.2.5 gyrY

```
int16_t DevLab_MPU6050::SensorData::gyrY
```

Definition at line 51 of file DevLab_MPU6050.h.

#### 5.3.2.6 gyrZ

```
int16_t DevLab_MPU6050::SensorData::gyrZ
```

Definition at line 52 of file DevLab_MPU6050.h.

#### 5.3.2.7 temperatureC

```
float DevLab_MPU6050::SensorData::temperatureC
```

Definition at line 55 of file DevLab_MPU6050.h.

#### 5.3.2.8 tempRaw

```
int16_t DevLab_MPU6050::SensorData::tempRaw
```

Definition at line 54 of file DevLab_MPU6050.h.

The documentation for this struct was generated from the following file:

- /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.h

# Chapter 6

# File Documentation

## 6.1 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/basic_Interrupt/basic_Interrupt.ino File Reference

## 6.2 basic_Interrupt.ino

Go to the documentation of this file.

```
00001 #include <DevLab_MPU6050.h>
00002
00003 #define SDA_PIN 6
00004 #define SCL_PIN 7
00005 #define MPU6050_ADDR 0x68
00006 #define INT_PIN 4
00007
00008 volatile uint32_t irqCount = 0;
00009
00010 DevLab_MPU6050 imu(Wire, MPU6050_ADDR);
00011 SensorData data;
00012 interruptPinConfig intCfg{
00013   0,0,1,0,0,0,0
00014 };
00015
00016 void IRAM_ATTR onMPUInterrupt() {
00017   irqCount++;
00018 }
00019
00020 void setup() {
00021   Serial.begin(115200);
00022   delay(1000);
00023
00024   pinMode(INT_PIN, INPUT);
00025   attachInterrupt(digitalPinToInterrupt(INT_PIN), onMPUInterrupt, RISING);
00026
00027   Serial.println("Inicializando MPU6050...");
00028
00029   if (!imu.begin(SDA_PIN, SCL_PIN, 400000)) {
00030     Serial.println("ERROR: MPU6050 no encontrado.");
00031     while (1) {
00032       delay(1000);
00033     }
00034   }
00035
00036   imu.setIntPinCfg(intCfg);
00037
00038   imu.enableInterrupt(INT_DATA_READY);
00039
00040
00041   Serial.println("MPU6050 inicializado con interrupcion DATA_RDY.");
00042 }
00043
00044 void loop() {
00045   if (imu.readData(data)) {
```

```
00046     Serial.print("aX = ");
00047     Serial.print(data.accX);
00048     Serial.print(" | aY = ");
00049     Serial.print(data.accY);
00050     Serial.print(" | aZ = ");
00051     Serial.print(data.accZ);
00052
00053     Serial.print(" | temp = ");
00054     Serial.print(data.temperatureC, 2);
00055
00056     Serial.print(" | gX = ");
00057     Serial.print(data.gyrX);
00058     Serial.print(" | gY = ");
00059     Serial.print(data.gyrY);
00060     Serial.print(" | gZ = ");
00061     Serial.print(data.gyrZ);
00062
00063     Serial.print(" | irqCount = ");
00064     Serial.println(irqCount);
00065     uint8_t intStatus = imu.interruptStatus();
00066     Serial.println(intStatus,HEX);
00067   } else {
00068     Serial.println("ERROR: fallo en la lectura del MPU6050.");
00069   }
00070
00071   delay(500);
00072 }
```

## 6.3 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/BasicRead/BasicRead.ino File Reference

## 6.4 BasicRead.ino

Go to the documentation of this file.

```
00001 #include <DevLab_MPU6050.h>
00002
00003 #define SDA_PIN 6
00004 #define SCL_PIN 7
00005 #define MPU6050_ADDR 0x68
00006
00007 DevLab_MPU6050 imu(Wire, MPU6050_ADDR);
00008
00009 SensorData data;
00010
00011 void setup() {
00012   Serial.begin(115200);
00013   delay(1000);
00014
00015   Serial.println("Inicializando MPU6050...");
00016
00017   if (!imu.begin(SDA_PIN, SCL_PIN, 400000)) {
00018     Serial.println("ERROR: MPU6050 no encontrado.");
00019     while (1) {
00020       delay(1000);
00021     }
00022   }
00023
00024   Serial.println("MPU6050 inicializado correctamente.");
00025 }
00026
00027 void loop() {
00028   if (imu.readData(data)) {
00029     Serial.print("aX = ");
00030     Serial.print(data.accX / 16384.0f);
00031     Serial.print(" | aY = ");
00032     Serial.print(data.accY / 16384.0f);
00033     Serial.print(" | aZ = ");
00034     Serial.print(data.accZ /4096.0f);
00035
00036     Serial.print(" | temp = ");
00037     Serial.print(data.temperatureC, 2);
00038
00039     Serial.print(" | gX = ");
00040     Serial.print(data.gyrX);
```

```
00041     Serial.print(" | gY = ");
00042     Serial.print(data.gyrY);
00043     Serial.print(" | gZ = ");
00044     Serial.println(data.gyrZ);
00045   } else {
00046     Serial.println("ERROR: fallo en la lectura del MPU6050.");
00047   }
00048
00049   delay(1000);
00050 }
```

## 6.5 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/I2CIdentify/I2CIdentify.ino File Reference

## 6.6 I2CIdentify.ino

Go to the documentation of this file.

```
00001 #include "DevLab_MPU6050.h"
00002
00003 #define SDA_PIN 6
00004 #define SCL_PIN 7
00005
00006 DevLab_MPU6050 imu;
00007
00008 uint8_t whoamireg = 0x00;
00009 #define DEFAULT_ADDRESS 0x68
00010
00011 void setup() {
00012   Serial.begin(115200);
00013
00014   delay(1000);
00015
00016   if (!imu.begin(SDA_PIN, SCL_PIN,400000)) {
00017     Serial.println("ERROR: MPU6050 initialization failed.");
00018
00019   }else{
00020     Serial.println("MPU6050 is working fine");
00021
00022   }
00023   // put your setup code here, to run once:
00024
00025 }
00026
00027 void loop() {
00028   // put your main code here, to run repeatedly:
00029   whoamireg = imu.WhoAmI();
00030   if(whoamireg == DEFAULT_ADDRESS){
00031       Serial.print("0x");
00032       Serial.println(whoamireg,HEX);
00033       delay(1000);
00034   }
00035
00036 }
```

## 6.7 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/MPU6050_BasicRead/MPU6050_↩ BasicRead.ino File Reference

## 6.8 MPU6050_BasicRead.ino

Go to the documentation of this file.

```
00001 #include <DevLab_MPU6050.h>
```

```
00002
00003 #define SDA_PIN 12
00004 #define SCL_PIN 22
00005 #define MPU6050_ADDR 0x68
00006
00007 DevLab_MPU6050 imu(Wire, MPU6050_ADDR);
00008 DevLab_MPU6050::SensorData data;
00009
00010 void setup() {
00011   Serial.begin(115200);
00012   delay(1000);
00013
00014   Serial.println("Inicializando MPU6050...");
00015
00016   if (!imu.begin(SDA_PIN, SCL_PIN, 400000)) {
00017     Serial.println("ERROR: MPU6050 no encontrado.");
00018     while (1) {
00019       delay(1000);
00020     }
00021   }
00022
00023   Serial.println("MPU6050 inicializado correctamente.");
00024 }
00025
00026 void loop() {
00027   if (imu.readData(data)) {
00028     Serial.print("aX = ");
00029     Serial.print(data.accX);
00030     Serial.print(" | aY = ");
00031     Serial.print(data.accY);
00032     Serial.print(" | aZ = ");
00033     Serial.print(data.accZ);
00034
00035     Serial.print(" | temp = ");
00036     Serial.print(data.temperatureC, 2);
00037
00038     Serial.print(" | gX = ");
00039     Serial.print(data.gyrX);
00040     Serial.print(" | gY = ");
00041     Serial.print(data.gyrY);
00042     Serial.print(" | gZ = ");
00043     Serial.println(data.gyrZ);
00044   } else {
00045     Serial.println("ERROR: fallo en la lectura del MPU6050.");
00046   }
00047
00048   delay(1000);
00049 }
```

## 6.9 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/MPU6050_ConfigSweep/MPU6050_↩ ConfigSweep.ino File Reference

## 6.10 MPU6050_ConfigSweep.ino

Go to the documentation of this file.

```
00001 /*
00002   MPU6050_ConfigSweep.ino
00003
00004   Sweeps every combination of AccelScale, GyroScale and DLPF, applies
00005   it to the MPU6050 and prints one reading per combination as CSV over
00006   Serial. Useful for characterizing how each setting affects the raw
00007   output (e.g. importing the log into a spreadsheet or a script for
00008   analysis).
00009
00010   CSV columns:
00011   accelScale,gyroScale,dlpf,smplrtDiv,accX,accY,accZ,gyrX,gyrY,gyrZ,tempC,
00012   gyroBW_Hz,outputRate_Hz,nyquist_Hz,dlpf_below_nyquist
00013 */
00014
00015 #include <DevLab_MPU6050.h>
00016
00017 #define SDA_PIN 6
00018 #define SCL_PIN 7
00019 #define MPU6050_ADDR 0x68
```

```
00020
00021 DevLab_MPU6050 imu(Wire, MPU6050_ADDR);
00022 DevLab_MPU6050::SensorData data;
00023
00024 const uint8_t ACCEL_SCALES[]  = {0, 1, 2, 3};
00025 const uint8_t GYRO_SCALES[]   = {0, 1, 2, 3};
00026 const uint8_t DLPF_SETTINGS[] = {0, 1, 2, 3, 4, 5, 6};
00027
00028 // Element counts computed explicitly -- sizeof(array) on a uint8_t[]
00029 // happens to equal the element count only because uint8_t is 1 byte.
00030 const uint8_t N_ACCEL = sizeof(ACCEL_SCALES)  / sizeof(ACCEL_SCALES[0]);
00031 const uint8_t N_GYRO  = sizeof(GYRO_SCALES)   / sizeof(GYRO_SCALES[0]);
00032 const uint8_t N_DLPF  = sizeof(DLPF_SETTINGS) / sizeof(DLPF_SETTINGS[0]);
00033
00034 const uint8_t CLK_SOURCE_PLL_GYRO_X = 1; // stable reference, recommended by Invensense
00035
00036 // DLPF_CFG bandwidths in Hz (MPU-6000/6050 register map, DLPF_CFG table).
00037 const float ACCEL_BW_HZ[] = {260.0f, 184.0f, 94.0f, 44.0f, 21.0f, 10.0f, 5.0f};
00038 const float GYRO_BW_HZ[]  = {256.0f, 188.0f, 98.0f, 42.0f, 20.0f, 10.0f, 5.0f};
00039
00040 // Nyquist must be at least this many times the DLPF corner frequency.
00041 const float NYQUIST_MARGIN = 1.2f;
00042
00043 // Number of sample periods to wait after an AFS_SEL/FS_SEL change before
00044 // trusting readData() -- absorbs I2C transaction latency and ADC
00045 // conversion jitter so we don't sample mid-transition.
00046 const uint8_t SCALE_SETTLE_PERIODS = 5;
00047
00048 // How many full sweeps to run back-to-back (set from setup(), or change
00049 // here and call sweepAndPrint() once per iteration from setup()/loop()).
00050 const uint16_t N_REPEATS = 100;
00051
00052 // Internal sampling rate feeding SMPLRT_DIV: 8kHz when the DLPF is
00053 // bypassed (DLPF_CFG=0), 1kHz for any other DLPF setting.
00054 float internalFsHz(uint8_t dlpf) {
00055   return (dlpf == 0) ? 8000.0f : 1000.0f;
00056 }
00057
00058 // SMPLRT_DIV per DLPF_CFG, computed once at startup. Chosen as the
00059 // LARGEST divisor (lowest output rate, gentlest on I2C traffic) that
00060 // still keeps Nyquist >= NYQUIST_MARGIN * corner.
00061 uint8_t g_smplrtDivByDlpf[7];
00062
00063 void computeSmplrtDivTable() {
00064   for (uint8_t d = 0; d < N_DLPF; d++) {
00065     uint8_t dlpf = DLPF_SETTINGS[d];
00066     float fsBase = internalFsHz(dlpf);
00067     float cornerHz = min(ACCEL_BW_HZ[d], GYRO_BW_HZ[d]);
00068
00069     float maxRatio = fsBase / (2.0f * NYQUIST_MARGIN * cornerHz);
00070     long divCandidate = (long)floor(maxRatio - 1.0f);
00071
00072     if (divCandidate < 0)   divCandidate = 0;
00073     if (divCandidate > 255) divCandidate = 255;
00074
00075     g_smplrtDivByDlpf[d] = (uint8_t)divCandidate;
00076   }
00077 }
00078
00079 float samplePeriodMs(uint8_t dIndex) {
00080   uint8_t dlpf = DLPF_SETTINGS[dIndex];
00081   float fsBase = internalFsHz(dlpf);
00082   uint8_t div = g_smplrtDivByDlpf[dIndex];
00083   return 1000.0f * (1 + div) / fsBase;
00084 }
00085
00086 // Time for the DLPF's single-pole low-pass to settle (~5 time constants,
00087 // tau = 1/(2*pi*BW)), using the tighter of the accel/gyro corner
00088 // frequencies, plus one sample period so the settled value has actually
00089 // been latched into the output registers.
00090 uint16_t dlpfSettleMs(uint8_t dIndex) {
00091   float bwHz = min(ACCEL_BW_HZ[dIndex], GYRO_BW_HZ[dIndex]);
00092   float tauMs = 1000.0f / (2.0f * PI * bwHz);
00093   float settleMs = 5.0f * tauMs + samplePeriodMs(dIndex);
00094   return (uint16_t)ceil(settleMs);
00095 }
00096
00097 // Time to wait after an AFS_SEL/FS_SEL change: several sample periods.
00098 uint16_t scaleSettleMs(uint8_t dIndex) {
00099   float settleMs = SCALE_SETTLE_PERIODS * samplePeriodMs(dIndex);
00100   return (uint16_t)ceil(settleMs);
00101 }
00102
00103 void sweepAndPrint() {
00104   // --- Repeat-call fix ------------------------------------------------
00105   // Explicitly (re)apply and settle the FIRST combination of the sweep
00106   // (AFS_SEL=0, FS_SEL=0, DLPF_CFG=0) before entering the nested loops.
```

```
00107   // Without this, if sweepAndPrint() is called back-to-back in a loop,
00108   // this first reading follows immediately after the LAST combination
00109   // of the previous call (AFS_SEL=3, DLPF_CFG=6) with no settle at all,
00110   // and can carry over the previous call's scale (see header note).
00111   // dlpfSettleMs() is used here (not scaleSettleMs()) because it's the
00112   // longer of the two and covers both the DLPF and the scale settling.
00113   imu.setDLPF(DLPF_SETTINGS[0]);
00114   imu.sampleRateDiv(g_smplrtDivByDlpf[0]);
00115   imu.setAccelScale(ACCEL_SCALES[0]);
00116   imu.setGyroScale(GYRO_SCALES[0]);
00117   delay(dlpfSettleMs(0));
00118   // ------------------------------------------------------------------------
00119
00120   for (uint8_t d = 0; d < N_DLPF; d++) {
00121     uint8_t dlpf = DLPF_SETTINGS[d];
00122     uint8_t div = g_smplrtDivByDlpf[d];
00123
00124     imu.setDLPF(dlpf);
00125     imu.sampleRateDiv(div);
00126     delay(dlpfSettleMs(d));
00127
00128     float fsBase = internalFsHz(dlpf);
00129     float outputRateHz = fsBase / (1 + div);
00130     float nyquistHz = outputRateHz / 2.0f;
00131     float gyroBw = GYRO_BW_HZ[d];
00132     float cornerHz = min(ACCEL_BW_HZ[d], GYRO_BW_HZ[d]);
00133     bool belowNyquist = (cornerHz <= nyquistHz);
00134
00135     uint16_t scaleDelayMs = scaleSettleMs(d);
00136
00137     for (uint8_t a = 0; a < N_ACCEL; a++) {
00138       imu.setAccelScale(ACCEL_SCALES[a]);
00139
00140       for (uint8_t g = 0; g < N_GYRO; g++) {
00141         imu.setGyroScale(GYRO_SCALES[g]);
00142
00143         delay(scaleDelayMs);
00144
00145         Serial.print(ACCEL_SCALES[a]); Serial.print(',');
00146         Serial.print(GYRO_SCALES[g]);  Serial.print(',');
00147         Serial.print(dlpf);            Serial.print(',');
00148         Serial.print(div);             Serial.print(',');
00149
00150         if (imu.readData(data)) {
00151           Serial.print(data.accX); Serial.print(',');
00152           Serial.print(data.accY); Serial.print(',');
00153           Serial.print(data.accZ); Serial.print(',');
00154           Serial.print(data.gyrX); Serial.print(',');
00155           Serial.print(data.gyrY); Serial.print(',');
00156           Serial.print(data.gyrZ); Serial.print(',');
00157           Serial.print(data.temperatureC, 2); Serial.print(',');
00158
00159           Serial.print(gyroBw, 1);          Serial.print(',');
00160           Serial.print(outputRateHz, 2);    Serial.print(',');
00161           Serial.print(nyquistHz, 2);       Serial.print(',');
00162           Serial.println(belowNyquist ? "OK" : "ALIAS_RISK");
00163         } else {
00164           Serial.println("READ_ERROR");
00165         }
00166       }
00167     }
00168   }
00169 }
00170
00171 void setup() {
00172   Serial.begin(115200);
00173   delay(1000);
00174
00175   Serial.println("Inicializando MPU6050...");
00176
00177   if (!imu.begin(SDA_PIN, SCL_PIN, 400000)) {
00178     Serial.println("ERROR: MPU6050 no encontrado.");
00179     while (1) {
00180       delay(1000);
00181     }
00182   }
00183
00184   imu.setClkSource(CLK_SOURCE_PLL_GYRO_X);
00185   computeSmplrtDivTable();
00186
00187   Serial.println("MPU6050 inicializado correctamente.");
00188   Serial.println("Tabla SMPLRT_DIV calculada por DLPF_CFG (Nyquist >= 1.2x corner):");
00189   for (uint8_t d = 0; d < N_DLPF; d++) {
00190     Serial.print("#  DLPF_CFG=");
00191     Serial.print(DLPF_SETTINGS[d]);
00192     Serial.print(" -> SMPLRT_DIV=");
00193     Serial.println(g_smplrtDivByDlpf[d]);
```

```
00194   }
00195
00196   Serial.print("Repeticiones del sweep completo: ");
00197   Serial.println(N_REPEATS);
00198
00199   Serial.println("Iniciando sweep de configuraciones...");
00200
      Serial.println("accelScale,gyroScale,dlpf,smplrtDiv,accX,accY,accZ,gyrX,gyrY,gyrZ,tempC,gyroBW_Hz,outputRate_Hz,nyquist_
00201
00202   for (uint16_t i = 0; i < N_REPEATS; i++) {
00203     sweepAndPrint();
00204   }
00205
00206   Serial.println("Sweep completo.");
00207 }
00208
00209 void loop() {
00210   // Nothing to do here; the sweep runs once in setup().
00211 }
```

## 6.11 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/examples/I2C/MPU6050_Interrupt/MPU6050_↩ Interrupt.ino File Reference

## 6.12 MPU6050_Interrupt.ino

Go to the documentation of this file.

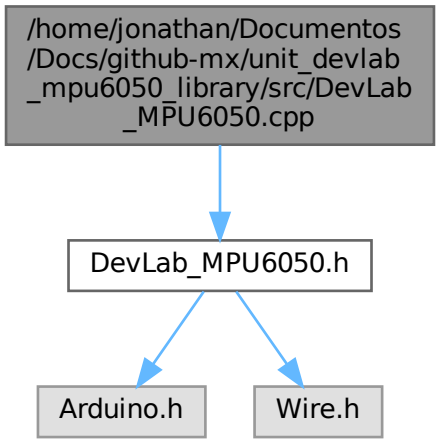
```
00001 #include <DevLab_MPU6050.h>
00002
00003 #define SDA_PIN 12
00004 #define SCL_PIN 22
00005 #define MPU6050_ADDR 0x68
00006 #define INT_PIN 4
00007
00008 volatile uint32_t irqCount = 0;
00009
00010 DevLab_MPU6050 imu(Wire, MPU6050_ADDR);
00011 DevLab_MPU6050::SensorData data;
00012
00013 void IRAM_ATTR onMPUInterrupt() {
00014   irqCount++;
00015 }
00016
00017 void setup() {
00018   Serial.begin(115200);
00019   delay(1000);
00020
00021   pinMode(INT_PIN, INPUT);
00022   attachInterrupt(digitalPinToInterrupt(INT_PIN), onMPUInterrupt, RISING);
00023
00024   Serial.println("Inicializando MPU6050...");
00025
00026   if (!imu.begin(SDA_PIN, SCL_PIN, 400000)) {
00027     Serial.println("ERROR: MPU6050 no encontrado.");
00028     while (1) {
00029       delay(1000);
00030     }
00031   }
00032
00033   Wire.beginTransmission(MPU6050_ADDR);
00034   Wire.write(0x37);
00035   Wire.write(0x00);
00036   Wire.endTransmission();
00037
00038   Wire.beginTransmission(MPU6050_ADDR);
00039   Wire.write(0x38);
00040   Wire.write(0x01);
00041   Wire.endTransmission();
00042
00043   Serial.println("MPU6050 inicializado con interrupcion DATA_RDY.");
00044 }
00045
00046 void loop() {
00047   if (imu.readData(data)) {
00048     Serial.print("aX = ");
```

```
00049     Serial.print(data.accX);
00050     Serial.print(" | aY = ");
00051     Serial.print(data.accY);
00052     Serial.print(" | aZ = ");
00053     Serial.print(data.accZ);
00054
00055     Serial.print(" | temp = ");
00056     Serial.print(data.temperatureC, 2);
00057
00058     Serial.print(" | gX = ");
00059     Serial.print(data.gyrX);
00060     Serial.print(" | gY = ");
00061     Serial.print(data.gyrY);
00062     Serial.print(" | gZ = ");
00063     Serial.print(data.gyrZ);
00064
00065     Serial.print(" | irqCount = ");
00066     Serial.println(irqCount);
00067   } else {
00068     Serial.println("ERROR: fallo en la lectura del MPU6050.");
00069   }
00070
00071   delay(500);
00072 }
```

## 6.13 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_↩ mpu6050_library/src/DevLab_MPU6050.cpp File Reference

```
#include "DevLab_MPU6050.h"
```

Include dependency graph for DevLab_MPU6050.cpp:




## 6.14 DevLab_MPU6050.cpp

Go to the documentation of this file.

```
00001 /*
00002   DevLab_MPU6050.cpp
00003
00004   Implementation file for the DevLab_MPU6050 Arduino library.
00005   This library communicates with the MPU6050 IMU using direct
00006   register access over I2C.
```

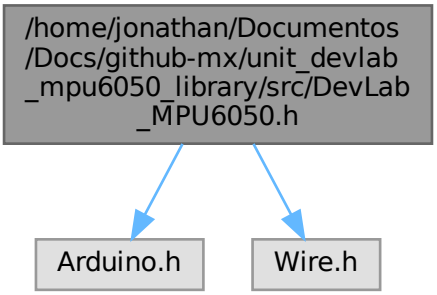
```
00007
00008   Author:
00009   UNIT Electronics
00010 */
00011
00012 #include "DevLab_MPU6050.h"
00013
00014 DevLab_MPU6050::DevLab_MPU6050(TwoWire &wire, uint8_t address) {
00015   _wire = &wire;
00016   _address = address;
00017   _accelScale = 0;
00018   _gyroScale = 0;
00019 }
00020
00021 bool DevLab_MPU6050::begin(uint8_t sdaPin, uint8_t sclPin, uint32_t clock) {
00022   _wire->begin(sdaPin, sclPin);
00023   _wire->setClock(clock);
00024
00025   delay(50);
00026
00027   uint8_t whoAmI = WhoAmI();
00028   if (whoAmI != 0x68) {
00029     return false;
00030   }
00031
00032   writeRegister8(REG_PWR_MGMT_1, 0x01); // Wake up the MPU6050
00033   delay(10);
00034
00035   writeRegister8(REG_SMPLRT_DIV, 0x07);
00036   writeRegister8(REG_CONFIG, 0x06);
00037   writeRegister8(REG_GYRO_CONFIG, 0x08);
00038   writeRegister8(REG_ACCEL_CONFIG, 0x10);
00039
00040   return true;
00041 }
00042
00043 uint8_t DevLab_MPU6050::WhoAmI() {
00044   return readRegister8(REG_WHO_AM_I);
00045 }
00046
00047 bool DevLab_MPU6050::readData(SensorData &dataOut) {
00048   _wire->beginTransmission(_address);
00049   _wire->write(REG_ACCEL_XOUT_H);
00050   if (_wire->endTransmission(false) != 0) {
00051     return false;
00052   }
00053
00054   _wire->requestFrom(_address, (uint8_t)14, (uint8_t)true);
00055
00056   if (_wire->available() < 14) {
00057     return false;
00058   }
00059
00060   dataOut.accX = (int16_t)(_wire->read() << 8 | _wire->read());
00061   dataOut.accY = (int16_t)(_wire->read() << 8 | _wire->read());
00062   dataOut.accZ = (int16_t)(_wire->read() << 8 | _wire->read());
00063
00064   dataOut.tempRaw = (int16_t)(_wire->read() << 8 | _wire->read());
00065   dataOut.temperatureC = dataOut.tempRaw / 340.0f + 36.53f;
00066
00067   dataOut.gyrX = (int16_t)(_wire->read() << 8 | _wire->read());
00068   dataOut.gyrY = (int16_t)(_wire->read() << 8 | _wire->read());
00069   dataOut.gyrZ = (int16_t)(_wire->read() << 8 | _wire->read());
00070
00071   return true;
00072 }
00073
00074 void DevLab_MPU6050::setDLPF(uint8_t dlpSetting) {
00075   if(dlpSetting > 6) {
00076     return;
00077   }
00078   uint8_t currentConfig = readRegister8(REG_CONFIG);
00079   writeRegister8(REG_CONFIG,(currentConfig & 0xF8) | (dlpSetting & 0x07));
00080 }
00081
00082 void DevLab_MPU6050::setAccelScale(uint8_t scale) {
00083
00084   _accelScale = scale;
00085
00086     if(scale > 3) {
00087     return;
00088   }
00089   uint8_t currentConfig = readRegister8(REG_ACCEL_CONFIG);
00090   writeRegister8(REG_ACCEL_CONFIG, (currentConfig & 0xE7) | ((scale & 0x03) << 3));
00091 }
00092
00093 void DevLab_MPU6050::setGyroScale(uint8_t scale) {
```

```
00094   _gyroScale = scale;
00095
00096   if(scale > 3) {
00097     return;
00098   }
00099   uint8_t currentConfig = readRegister8(REG_GYRO_CONFIG);
00100   writeRegister8(REG_GYRO_CONFIG, (currentConfig & 0xE7) | ((scale & 0x03) << 3));
00101 }
00102
00103 void DevLab_MPU6050::sampleRateDiv(uint8_t div) {
00104   writeRegister8(REG_SMPLRT_DIV, div);
00105 }
00106
00107 void DevLab_MPU6050::setClkSource(uint8_t clkSource) {
00108   if(clkSource > 7) {
00109     return;
00110   }
00111   uint8_t currentConfig = readRegister8(REG_PWR_MGMT_1);
00112   writeRegister8(REG_PWR_MGMT_1, (currentConfig & 0xF8) | (clkSource & 0x07));
00113 }
00114
00115 void DevLab_MPU6050::setIntPinCfg(uint8_t intPinCfg) {
00116   writeRegister8(REG_INT_PIN_CFG, intPinCfg);
00117 }
00118
00119 void DevLab_MPU6050::setIntPinCfg(const interruptPinConfig &cfg) {
00120   uint8_t value = (cfg.activeLow            << 7) |
00121                   (cfg.openDrain            << 6) |
00122                   (cfg.latchUntilCleared    << 5) |
00123                   (cfg.clearOnRead          << 4) |
00124                   (cfg.fsyncActiveLow       << 3) |
00125                   (cfg.enableFSyncInterrupt << 2) |
00126                   (cfg.enableI2CBypass      << 1);
00127   writeRegister8(REG_INT_PIN_CFG, value);
00128 }
00129
00130 void DevLab_MPU6050::enableInterrupt(uint8_t sources) {
00131   uint8_t currentConfig = readRegister8(REG_INT_ENABLE);
00132   writeRegister8(REG_INT_ENABLE, currentConfig | sources);
00133 }
00134
00135 void DevLab_MPU6050::disableInterrupt(uint8_t sources) {
00136   uint8_t currentConfig = readRegister8(REG_INT_ENABLE);
00137   writeRegister8(REG_INT_ENABLE, currentConfig & ~sources);
00138 }
00139
00140 uint8_t DevLab_MPU6050::interruptStatus() {
00141   return readRegister8(REG_INT_STATUS);
00142 }
00143
00144
00145
00146
00147
00148 /******* Private Utility functions for register access *******/
00149 void DevLab_MPU6050::writeRegister8(uint8_t reg, uint8_t value) {
00150   _wire->beginTransmission(_address);
00151   _wire->write(reg);
00152   _wire->write(value);
00153   _wire->endTransmission();
00154 }
00155
00156 uint8_t DevLab_MPU6050::readRegister8(uint8_t reg) {
00157   _wire->beginTransmission(_address);
00158   _wire->write(reg);
00159   _wire->endTransmission(false);
00160
00161   _wire->requestFrom(_address, (uint8_t)1, (uint8_t)true);
00162   if (_wire->available() < 1) {
00163     return 0;
00164   }
00165
00166   return _wire->read();
00167 }
00168
00169 int16_t DevLab_MPU6050::readRegister16(uint8_t reg) {
00170   _wire->beginTransmission(_address);
00171   _wire->write(reg);
00172   _wire->endTransmission(false);
00173
00174   _wire->requestFrom(_address, (uint8_t)2, (uint8_t)true);
00175   if (_wire->available() < 2) {
00176     return 0;
00177   }
00178
00179   return (int16_t)((uint16_t)_wire->read() << 8 | (uint16_t)_wire->read());
00180 }
```

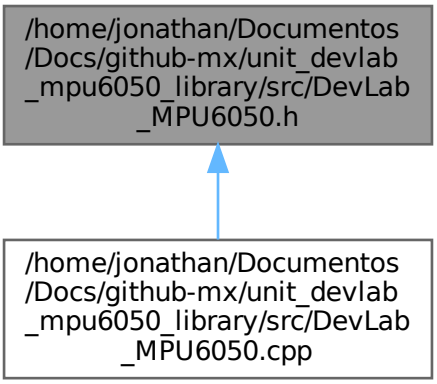
# 6.15 /home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.h File Reference

```
#include <Arduino.h>
#include <Wire.h>
```

Include dependency graph for DevLab_MPU6050.h:




This graph shows which files directly or indirectly include this file:




## Classes

- class DevLab_MPU6050
- struct DevLab_MPU6050::SensorData
- struct DevLab_MPU6050::interruptPinConfig

**Macros**

- #define MPU6050_ADDRESS_AD0_LOW 0x68
- #define MPU6050_ADDRESS_AD0_HIGH 0x69
- #define REG_INT_PIN_CFG 0x37
- #define REG_INT_ENABLE 0x38
- #define REG_INT_STATUS 0x3A
- #define REG_SMPLRT_DIV 0x19
- #define REG_CONFIG 0x1A
- #define REG_GYRO_CONFIG 0x1B
- #define REG_ACCEL_CONFIG 0x1C
- #define REG_ACCEL_XOUT_H 0x3B
- #define REG_TEMP_OUT_H 0x41
- #define REG_GYRO_XOUT_H 0x43
- #define REG_PWR_MGMT_1 0x6B
- #define REG_WHO_AM_I 0x75

**Typedefs**

- using SensorData = DevLab_MPU6050::SensorData
- using interruptPinConfig = DevLab_MPU6050::interruptPinConfig

**Variables**

- constexpr auto INT_DATA_READY = DevLab_MPU6050::INT_DATA_READY
- constexpr auto INT_I2C_MST = DevLab_MPU6050::INT_I2C_MST
- constexpr auto INT_FIFO_OVERFLOW = DevLab_MPU6050::INT_FIFO_OVERFLOW
- constexpr auto INT_MOTION = DevLab_MPU6050::INT_MOTION

## 6.15.1 Macro Definition Documentation

### 6.15.1.1 MPU6050_ADDRESS_AD0_HIGH

```
#define MPU6050_ADDRESS_AD0_HIGH 0x69
```

Definition at line 28 of file DevLab_MPU6050.h.

### 6.15.1.2 MPU6050_ADDRESS_AD0_LOW

```
#define MPU6050_ADDRESS_AD0_LOW 0x68
```

Definition at line 27 of file DevLab_MPU6050.h.

### 6.15.1.3 REG_ACCEL_CONFIG

```
#define REG_ACCEL_CONFIG 0x1C
```

Definition at line 36 of file DevLab_MPU6050.h.

#### 6.15.1.4 REG_ACCEL_XOUT_H

```
#define REG_ACCEL_XOUT_H 0x3B
```

Definition at line 37 of file DevLab_MPU6050.h.

#### 6.15.1.5 REG_CONFIG

```
#define REG_CONFIG 0x1A
```

Definition at line 34 of file DevLab_MPU6050.h.

#### 6.15.1.6 REG_GYRO_CONFIG

```
#define REG_GYRO_CONFIG 0x1B
```

Definition at line 35 of file DevLab_MPU6050.h.

#### 6.15.1.7 REG_GYRO_XOUT_H

```
#define REG_GYRO_XOUT_H 0x43
```

Definition at line 39 of file DevLab_MPU6050.h.

#### 6.15.1.8 REG_INT_ENABLE

```
#define REG_INT_ENABLE 0x38
```

Definition at line 31 of file DevLab_MPU6050.h.

#### 6.15.1.9 REG_INT_PIN_CFG

```
#define REG_INT_PIN_CFG 0x37
```

Definition at line 30 of file DevLab_MPU6050.h.

#### 6.15.1.10 REG_INT_STATUS

```
#define REG_INT_STATUS 0x3A
```

Definition at line 32 of file DevLab_MPU6050.h.

#### 6.15.1.11 REG_PWR_MGMT_1

```
#define REG_PWR_MGMT_1 0x6B
```

Definition at line 40 of file DevLab_MPU6050.h.

#### 6.15.1.12 REG_SMPLRT_DIV

```
#define REG_SMPLRT_DIV 0x19
```

Definition at line 33 of file DevLab_MPU6050.h.

#### 6.15.1.13 REG_TEMP_OUT_H

```
#define REG_TEMP_OUT_H 0x41
```

Definition at line 38 of file DevLab_MPU6050.h.

#### 6.15.1.14 REG_WHO_AM_I

```
#define REG_WHO_AM_I 0x75
```

Definition at line 41 of file DevLab_MPU6050.h.

### 6.15.2 Typedef Documentation

#### 6.15.2.1 interruptPinConfig

```
using interruptPinConfig = DevLab_MPU6050::interruptPinConfig
```

Definition at line 178 of file DevLab_MPU6050.h.

#### 6.15.2.2 SensorData

```
using SensorData = DevLab_MPU6050::SensorData
```

Definition at line 177 of file DevLab_MPU6050.h.

### 6.15.3 Variable Documentation

#### 6.15.3.1 INT_DATA_READY

```
auto INT_DATA_READY = DevLab_MPU6050::INT_DATA_READY  [constexpr]
```

Definition at line 179 of file DevLab_MPU6050.h.

#### 6.15.3.2 INT_FIFO_OVERFLOW

```
auto INT_FIFO_OVERFLOW = DevLab_MPU6050::INT_FIFO_OVERFLOW  [constexpr]
```

Definition at line 181 of file DevLab_MPU6050.h.

#### 6.15.3.3 INT_I2C_MST

```
auto INT_I2C_MST = DevLab_MPU6050::INT_I2C_MST  [constexpr]
```

Definition at line 180 of file DevLab_MPU6050.h.

#### 6.15.3.4 INT_MOTION

```
auto INT_MOTION = DevLab_MPU6050::INT_MOTION  [constexpr]
```

Definition at line 182 of file DevLab_MPU6050.h.

## 6.16 DevLab_MPU6050.h

Go to the documentation of this file.

```
00001 /*
00002   DevLab_MPU6050.h
00003
00004   Arduino library for the MPU6050 6-axis IMU
00005   using direct register communication over I2C.
00006
00007   Features:
00008   - Accelerometer reading
00009   - Gyroscope reading
00010   - Temperature reading
00011   - Direct register access
00012   - Compatible with ESP32 and Arduino-compatible boards
00013
00014   Author:
00015   UNIT Electronics,Jonathan Mejorado Lopez
00016
00017   License:
00018   MIT License
00019 */
00020
00021 #ifndef DEVLAB_MPU6050_H
00022 #define DEVLAB_MPU6050_H
00023
00024 #include <Arduino.h>
00025 #include <Wire.h>
00026
00027 #define MPU6050_ADDRESS_AD0_LOW  0x68
00028 #define MPU6050_ADDRESS_AD0_HIGH 0x69
00029
00030 #define REG_INT_PIN_CFG      0x37
00031 #define REG_INT_ENABLE       0x38
00032 #define REG_INT_STATUS       0x3A
00033 #define REG_SMPLRT_DIV       0x19
00034 #define REG_CONFIG           0x1A
00035 #define REG_GYRO_CONFIG      0x1B
00036 #define REG_ACCEL_CONFIG     0x1C
00037 #define REG_ACCEL_XOUT_H     0x3B
00038 #define REG_TEMP_OUT_H       0x41
00039 #define REG_GYRO_XOUT_H      0x43
00040 #define REG_PWR_MGMT_1       0x6B
00041 #define REG_WHO_AM_I         0x75
00042
00043 class DevLab_MPU6050 {
00044 public:
00045   struct SensorData {
00046     int16_t accX;
00047     int16_t accY;
00048     int16_t accZ;
00049
00050     int16_t gyrX;
00051     int16_t gyrY;
00052     int16_t gyrZ;
00053
00054     int16_t tempRaw;
00055     float temperatureC;
00056   };
00057
```

```
00058   struct interruptPinConfig {
00059     bool activeLow           = false; // INT_LEVEL,      bit7
00060     bool openDrain           = false; // INT_OPEN,       bit6
00061     bool latchUntilCleared   = false; // LATCH_INT_EN,   bit5
00062     bool clearOnRead         = false; // INT_RD_CLEAR,   bit4
00063     bool fsyncActiveLow      = false; // FSYNC_INT_LEVEL,bit3
00064     bool enableFSyncInterrupt= false; // FSYNC_INT_EN,   bit2
00065     bool enableI2CBypass     = false; // I2C_BYPASS_EN,  bit1
00066   };
00067
00068   enum InterruptSource : uint8_t {
00069     INT_DATA_READY    = 0x01, // bit0, DATA_RDY_EN
00070     INT_I2C_MST       = 0x08, // bit3, I2C_MST_INT_EN
00071     INT_FIFO_OVERFLOW = 0x10, // bit4, FIFO_OFLOW_EN
00072     INT_MOTION        = 0x40, // bit6, MOT_INT_EN
00073   };
00074
00075   /**
00076    * @brief Construct a new DevLab_MPU6050 object.
00077    * @param wire The I2C bus to use.
00078    * @param address The I2C address of the MPU6050.
00079    */
00080   DevLab_MPU6050(TwoWire &wire = Wire, uint8_t address = MPU6050_ADDRESS_AD0_LOW);
00081
00082   /**
00083    * @brief Initialize the MPU6050.
00084    * @param sdaPin The SDA pin.
00085    * @param sclPin The SCL pin.
00086    * @param clock The I2C clock frequency.
00087    * @return True if the MPU6050 was initialized successfully, false otherwise.
00088    */
00089   bool begin(uint8_t sdaPin, uint8_t sclPin, uint32_t clock = 400000);
00090
00091   /**
00092    * @brief Read the sensor data.
00093    * @param dataOut The sensor data to read.
00094    * @return True if the data was read successfully, false otherwise.
00095    */
00096   bool readData(SensorData &dataOut);
00097
00098   /**
00099    * @brief Set the accelerometer scale.
00100    * @param scale The accelerometer scale to apply.
00101    */
00102   void setAccelScale(uint8_t scale);
00103
00104   /**
00105    * @brief Set the gyroscope scale.
00106    * @param scale The gyroscope scale to apply.
00107    */
00108   void setGyroScale(uint8_t scale);
00109   /* Special functions and setting ups for the MPU6050 */
00110
00111   /**
00112    * @brief Get the device ID.
00113    * @return The device ID.
00114    */
00115   uint8_t WhoAmI();
00116
00117   /**
00118    * @brief Set the Digital Low Pass Filter (DLPF) setting.
00119    * @param dlpSetting The DLPF setting to apply.
00120    */
00121   void setDLPF(uint8_t dlpSetting);
00122
00123   /**
00124    * @brief Set the sample rate divider.
00125    * @param div The sample rate divider to apply.
00126    */
00127   void sampleRateDiv(uint8_t div);
00128
00129   /**
00130    * @brief Set the clock source.
00131    * @param clkSource The clock source to apply.
00132    */
00133   void setClkSource(uint8_t clkSource);
00134
00135   /**
00136    * @brief Set the INT pin / bypass configuration register directly.
00137    * @param intPinCfg Raw byte to write to REG_INT_PIN_CFG.
00138    */
00139   void setIntPinCfg(uint8_t intPinCfg);
00140
00141   /**
00142    * @brief Set the INT pin / bypass configuration from a struct.
00143    * @param cfg The interrupt pin configuration to apply.
00144    */
```

```
00145   void setIntPinCfg(const interruptPinConfig &cfg);
00146
00147   /**
00148    * @brief Enable one or more interrupt sources (OR'd together), leaving others untouched.
00149    * @param sources Bitwise-OR of InterruptSource values.
00150    */
00151   void enableInterrupt(uint8_t sources);
00152
00153   /**
00154    * @brief Disable one or more interrupt sources (OR'd together), leaving others untouched.
00155    * @param sources Bitwise-OR of InterruptSource values.
00156    */
00157   void disableInterrupt(uint8_t sources);
00158
00159   /**
00160    * @brief Read the INT_STATUS register without side effects other than the hardware's own
      clear-on-read.
00161    * @return The interrupt status byte.
00162    */
00163   uint8_t interruptStatus();
00164
00165 private:
00166   TwoWire *_wire;
00167   uint8_t _address;
00168
00169   uint8_t _accelScale;
00170   uint8_t _gyroScale;
00171
00172   void writeRegister8(uint8_t reg, uint8_t value);
00173   uint8_t readRegister8(uint8_t reg);
00174   int16_t readRegister16(uint8_t reg);
00175 };
00176
00177 using SensorData = DevLab_MPU6050::SensorData;
00178 using interruptPinConfig = DevLab_MPU6050::interruptPinConfig;
00179 constexpr auto INT_DATA_READY    = DevLab_MPU6050::INT_DATA_READY;
00180 constexpr auto INT_I2C_MST       = DevLab_MPU6050::INT_I2C_MST;
00181 constexpr auto INT_FIFO_OVERFLOW = DevLab_MPU6050::INT_FIFO_OVERFLOW;
00182 constexpr auto INT_MOTION        = DevLab_MPU6050::INT_MOTION;
00183 #endif
```

# Index


/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples
Directory Reference, 8
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C
Directory Reference, 8
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/BasicRead
Directory Reference, 8
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/BasicRead/BasicRead.ino, 24
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/I2CIdentify
Directory Reference, 9
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/I2CIdentify/I2CIdentify.ino, 25
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_BasicRead
Directory Reference, 9
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_BasicRead/MPU6050_BasicRead.ino, 25
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_ConfigSweep
Directory Reference, 10
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_ConfigSweep/MPU6050_ConfigSweep.ino, 26
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_Interrupt
Directory Reference, 10
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/MPU6050_Interrupt/MPU6050_Interrupt.ino, 29
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/basic_Interrupt
Directory Reference, 7
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/examples/I2C/basic_Interrupt/basic_Interrupt.ino, 23
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src
Directory Reference, 11
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.cpp, 30
/home/jonathan/Documentos/Docs/github-mx/unit_devlab_mpu6050_library/src/DevLab_MPU6050.h, 33, 37

accX
DevLab_MPU6050::SensorData, 21
accY
DevLab_MPU6050::SensorData, 21
accZ
DevLab_MPU6050::SensorData, 22
activeLow
DevLab_MPU6050::interruptPinConfig, 20

begin
DevLab_MPU6050, 15

clearOnRead
DevLab_MPU6050::interruptPinConfig, 20

DevLab_MPU6050, 13
begin, 15
DevLab_MPU6050, 14
enableInterrupt, 15
INT_DATA_READY, 14
INT_FIFO_OVERFLOW, 14
INT_I2C_MST, 14
INT_MOTION, 14
interruptStatus, 16
readData, 16
sampleRateDiv, 16
setAccelScale, 17
setClkSource, 17
setDLPF, 17
setGyroScale, 18
setInterruptPinConfig, 18
WhoAmI, 19
DevLab_MPU6050.h
INT_DATA_READY, 36
INT_FIFO_OVERFLOW, 36
INT_I2C_MST, 36
INT_MOTION, 37
interruptPinConfig, 36
MPU6050_ADDRESS_AD0_HIGH, 34
MPU6050_ADDRESS_AD0_LOW, 34
REG_ACCEL_CONFIG, 34
REG_ACCEL_XOUT_H, 34
REG_CONFIG, 35
REG_GYRO_CONFIG, 35
REG_GYRO_XOUT_H, 35
REG_INT_ENABLE, 35
REG_INT_PIN_CFG, 35
REG_INT_STATUS, 35
REG_PWR_MGMT_1, 35
REG_SMPLRT_DIV, 35
REG_TEMP_OUT_H, 36
REG_WHO_AM_I, 36
SensorData, 36
DevLab_MPU6050::interruptPinConfig, 19
activeLow, 20
clearOnRead, 20
enableFSyncInterrupt, 20
enableI2CBypass, 20
fsyncActiveLow, 20
latchUntilCleared, 20
openDrain, 21
DevLab_MPU6050::SensorData, 21
accX, 21

accY, 21
accZ, 22
gyrX, 22
gyrY, 22
gyrZ, 22
temperatureC, 22
tempRaw, 22
disableInterrupt
DevLab_MPU6050, 15

enableFSyncInterrupt
DevLab_MPU6050::interruptPinConfig, 20
enableI2CBypass
DevLab_MPU6050::interruptPinConfig, 20
enableInterrupt
DevLab_MPU6050, 15

fsyncActiveLow
DevLab_MPU6050::interruptPinConfig, 20

gyrX
DevLab_MPU6050::SensorData, 22
gyrY
DevLab_MPU6050::SensorData, 22
gyrZ
DevLab_MPU6050::SensorData, 22

INT_DATA_READY
DevLab_MPU6050, 14
DevLab_MPU6050.h, 36
INT_FIFO_OVERFLOW
DevLab_MPU6050, 14
DevLab_MPU6050.h, 36
INT_I2C_MST
DevLab_MPU6050, 14
DevLab_MPU6050.h, 36
INT_MOTION
DevLab_MPU6050, 14
DevLab_MPU6050.h, 37
interruptPinConfig
DevLab_MPU6050.h, 36
InterruptSource
DevLab_MPU6050, 14
interruptStatus
DevLab_MPU6050, 16

latchUntilCleared
DevLab_MPU6050::interruptPinConfig, 20

MPU6050_ADDRESS_AD0_HIGH
DevLab_MPU6050.h, 34
MPU6050_ADDRESS_AD0_LOW
DevLab_MPU6050.h, 34

openDrain
DevLab_MPU6050::interruptPinConfig, 21

readData
DevLab_MPU6050, 16
REG_ACCEL_CONFIG
DevLab_MPU6050.h, 34
REG_ACCEL_XOUT_H
DevLab_MPU6050.h, 34
REG_CONFIG
DevLab_MPU6050.h, 35
REG_GYRO_CONFIG
DevLab_MPU6050.h, 35
REG_GYRO_XOUT_H
DevLab_MPU6050.h, 35
REG_INT_ENABLE
DevLab_MPU6050.h, 35
REG_INT_PIN_CFG
DevLab_MPU6050.h, 35
REG_INT_STATUS
DevLab_MPU6050.h, 35
REG_PWR_MGMT_1
DevLab_MPU6050.h, 35
REG_SMPLRT_DIV
DevLab_MPU6050.h, 35
REG_TEMP_OUT_H
DevLab_MPU6050.h, 36
REG_WHO_AM_I
DevLab_MPU6050.h, 36

sampleRateDiv
DevLab_MPU6050, 16
SensorData
DevLab_MPU6050.h, 36
setAccelScale
DevLab_MPU6050, 17
setClkSource
DevLab_MPU6050, 17
setDLPF
DevLab_MPU6050, 17
setGyroScale
DevLab_MPU6050, 18
setIntPinCfg
DevLab_MPU6050, 18

temperatureC
DevLab_MPU6050::SensorData, 22
tempRaw
DevLab_MPU6050::SensorData, 22

WhoAmI
DevLab_MPU6050, 19